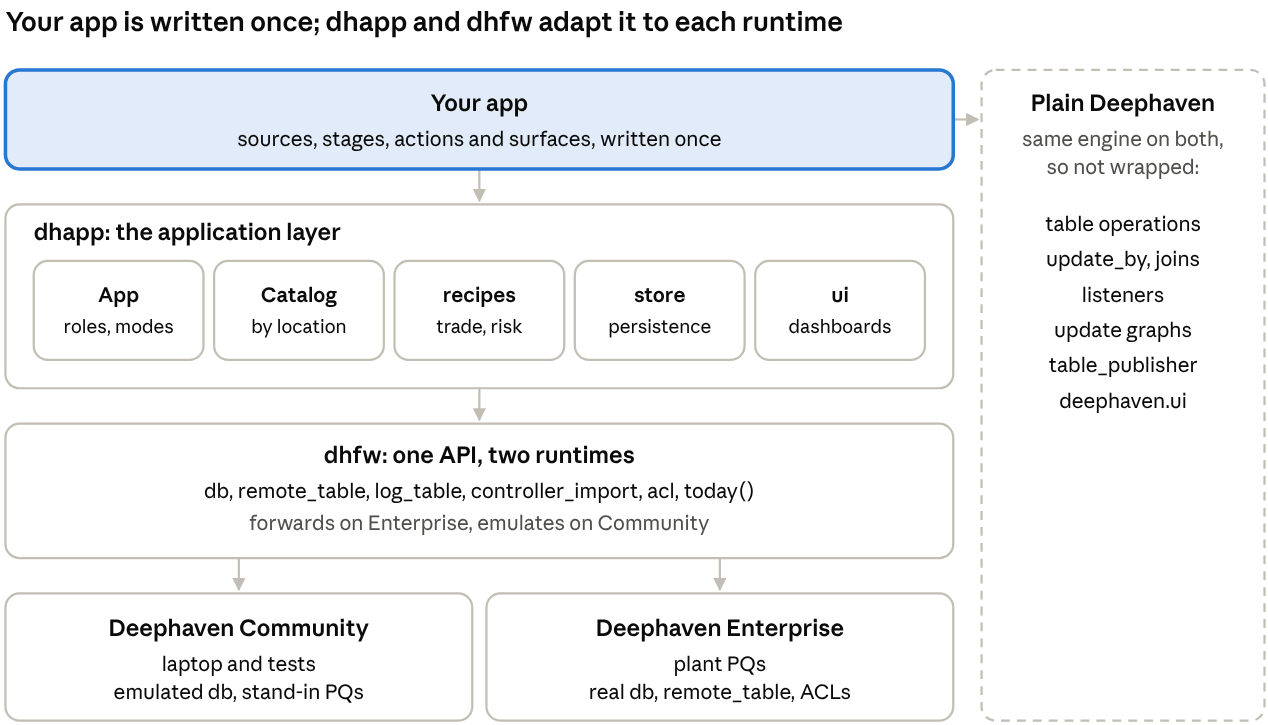
dhapp and dhfw: Framework Reference

Oct 21, 2026 · @Taylor

At a glance

Two thin layers on Deephaven let one codebase run on a laptop and on the plant. `dhapp` gives an app its structure (roles, modes, a location-aware catalog, recipes, storage, dashboards). `dhfw` hides the differences between Community and Enterprise. Both are in `dhapp/` and `dhfw/` in the repository. The engine's own API stays fully available.



layers · your app, dhapp, dhfw, and the two runtimes

Read it top to bottom. Your app calls `dhapp`, `dhapp` calls `dhfw`, and `dhfw` either forwards to Enterprise or emulates on Community. Table logic in stages calls plain Deephaven directly, on the right, because that engine is the same on both runtimes.

For how the engine itself works (cycles, threads, Barrage, replay), see the Programmer's Guide. This document is the reference for building on the two layers.

Quick start: a working app in five steps

Declare sources, write stages, add actions, surface tables, then start in a mode. This is the market-making demo in `examples/mm_app.py`, cut down.

```
from dhapp import App, Catalog, recipes, schemas
from dhapp.ui import dashboard

catalog = Catalog({"desk_limits": {"pq": "RiskLimits", "table":
"desk_limits", "plant": "ny"},
                   "desk_map":    {"db": ["Ref", "DeskMap"], "live": False}})
app = App("mm", catalog)

# 1. Sources: where rows come from
app.source("quotes", schemas.QUOTES, history=load_quotes)   # pushed live;
replayed in backtest
app.source("limits", catalog="desk_limits")                 # resolved by
location
app.source("desks",  catalog="desk_map")

# 2. Stages: pure functions; parameter names are table names
@app.stage
def orders(quotes):
    return
recipes.fill_at_touch(recipes.target_to_orders(recipes.ema_cross(quotes)))

@app.stage
def book(orders, quotes):
    return recipes.mark_to_market(recipes.positions(orders), quotes)

@app.stage
def risk(book, desks, limits):
    return recipes.risk_rollup(book, desks, limits)

# 3. Actions: the only per-cycle Python, supervised
@app.action(on="orders", services=["broker"])
def route(rows, broker):
    broker.send(rows)
app.service("broker", live=OrderGateway(), sim=RecordingBroker())

# 4. Surfaces: published fields, and the dashboard's panels
app.surface("book", "risk", "orders", "limits")
```

```
# 5. Start in a mode
app.start("live")                 # then app.feed("quotes", rows) from your
feed thread
mm_dashboard = dashboard(app, kpis={"risk": ["Gross", "PnL"]})
```

Run the full demo locally, then open `http://localhost:10000/iframe/widget/?name=mm_dashboard`:

```
.venv-server/Scripts/python server/run_server.py --app-dir examples/mm_app.d
```

The same declaration runs as a backtest with `app.start("backtest")` then `app.run_history()`, or as a test with `app.start("test")`, then `app.feed(...)` and `app.step()`.

Concepts

An app is a graph of named tables. The framework owns the wiring, the clock, the supervision and the publishing; you write the table logic.

Four roles

Role	Declared with	What it is	Rule
Source	`app.source(name, schema, history=..., live=..., catalog=...)`	rows entering the app	exactly one of: pushed (schema only), `live=` factory, `catalog=` name
Stage	`@app.stage`	`def name(input_a, input_b) -> table` or a dict of tables	pure: no side effects, no Python per row; use recipes or plain table ops
Action	`@app.action(on=table, services=[...])`	Python called with the rows added to `on` in each cycle	keep it short; exceptions are caught and recorded
Surface	`app.surface(*names)`	tables published as `<app>_<name>` fields and dashboard panels	publish results, not every intermediate

Stages run in dependency order, worked out from their parameter names. A name no source or stage produces is an error at `start()`.

Three modes

Mode	Clock	Pushed sources	Catalog and `live=` sources	Services
`live`	default, about 1 s per cycle	`app.feed(name, rows)` from any thread	live, ticking	`live`
`backtest`	your own clock, one checkpoint per time bucket	replayed from `history=` by `app.run_history()`	frozen (snapshot)	`sim`
`test`	your own clock, one checkpoint per `app.step()`	`app.feed(...)`, then `app.step()`	frozen (snapshot)	`sim`

Sources are frozen in backtest and test because one update graph cannot mix two ticking clocks. Everything the app owns runs on the backtest clock.

The catalog. App code uses logical names (`desk_limits`). The `Catalog` turns each into a table, based on where the app is running. Moving a table to another PQ, or deploying onto the plant that owns it, changes catalog entries or environment variables, never code.

Health. `app.health` is a live table with one row per action: `Calls`, `Errors`, `LastError`. It is published as `<app>_health` and shown on the dashboard's status panel.

Runtime. Everything that differs between Community and Enterprise (`db`, remote tables, logging, controller imports, ACLs, identity) goes through `dhfw`, which forwards on Enterprise and emulates on Community. Table operations, listeners and deephaven.ui are the same engine on both, so they are not wrapped.

API reference

`dhapp.App(name, catalog=None)`

Method	Purpose
`source(name, schema=None, *, live=None, history=None, catalog=None, append_only=True)`	declare a source; pushed sources are append-only by default (`append_only=False` keeps the blink table)
`@stage` / `stage(outputs=...)`	register a table function; returns a table (named after the function) or a dict of tables
`@action(on, services=(), name=None)`	`fn(rows: DataFrame, **services)`, called with each cycle's added rows
`service(name, *, live, sim)`	an object passed to actions, picked by mode
`surface(*names)`	publish as `<app>_<name>` and add a dashboard panel
`start(mode="live")`	build everything; `"backtest"` and `"test"` create the app's own update graph
`feed(source, table)`	push rows into a pushed source (thread-safe)
`step()`	backtest and test: one checkpoint, returning when it is finished
`run_history(bucket_ms=1000, ts="Timestamp")`	backtest: replay every `history=` source in time order; returns buckets, rows and seconds
`table(name)`, `tables`, `health`, `stop()`	read results; `stop()` removes the action listeners

`dhapp.Catalog(entries, plants=None)`

Entry	Resolves to
`{"db": [ns, table], "live": True}`	`db.live_table`, or `db.historical_table` when `live` is False
`{"pq": name, "table": t, "plant": p}`	a local global, `remote_table` on the same plant, or `for_remote_cluster` on another plant (see Deploying)
`override(name, table_or_factory)`	a fixture or local feed in any runtime

The catalog also has `resolve(name, subscribe_kwargs=None)`, and `binding(name)`, which gives the binding in words. `plants` takes `{name: {"url": ..., "key_file": ...}}`, or `user` with `password_env` in place of `key_file`.

`dhapp.recipes` (tables in, tables out)

Recipe	Input columns	Adds
`mid(quotes)`	Bid, Ask	Mid, Spread
`bars(trades, period="PT1m")`	Timestamp, Sym, Px, Qty	Open, High, Low, Close, Volume per Sym and Bin
`ema_cross(quotes, fast=5, slow=20, size=100, px="Mid")`	Sym, px (or Bid and Ask)	Fast, Slow, Target
`target_to_orders(signals, target="Target")`	Sym, Target	OrderQty (changes only)
`fill_at_touch(orders, fee_bps=0)`	OrderQty, Bid, Ask	FillPx, Notional, Fee
`positions(fills)`	Sym, OrderQty, Notional, Fee	Pos, Cost, Fees, Trades
`mark_to_market(positions, quotes, px="Mid")`	Sym	Mid, Mtm, PnL, Gross
`risk_rollup(book, desks, limits)`	Sym, Gross, PnL	per Desk: Gross, PnL, Names, Limit, Utilization, Breach

`dhapp.schemas`, `dhapp.store`, `dhapp.ui`

Name	Purpose
`QUOTES`, `TRADES`, `DESKS`, `LIMITS`; `validate(table, schema)`	standard shapes; sources with a schema are validated at start
`store.snapshot(t, ns, name)`	replace a user table with the table's current rows
`store.daily_partition(t, ns, name, date=None)`	write a day's partition (creates the schema once)
`store.intraday_log(t, ns, name)`	context manager: log added rows to Intraday while open
`store.state_table(ns, name, prototype, keys)`	a keyed, editable table (limits, overrides)
`ui.dashboard(app, kpis=None, formats=None, per_row=2)`	one panel per surface, KPI cards, a status panel
`ui.kpi_row`, `ui.table_panel`, `ui.status_panel`	building blocks for custom dashboards

`dhfw` (used underneath; usable directly)

`db` (Enterprise `Database` methods), `remote_table`, `resolve_uri`, `log_table`, `log_table_incremental`, `meta_import`, `exec_script`, `apply_acl`, `today()`, `effective_user()`, `runtime()`, `is_enterprise()`, `LIMITATIONS`.

Deploying

The app's code is the same everywhere; what changes per environment is a handful of environment variables and the catalog's answers.

Variable	Meaning	Laptop (Community)	Plant (Enterprise PQ)
`DHFW_PLANT`	this plant's name, compared with catalog `plant`	e.g. `ny`	the plant name, set in the PQ's environment
`DHFW_PQ`	this PQ's name, compared with catalog `pq`	the name of the PQ you are standing in for	the PQ's own name
`DHFW_RUNTIME`	force `community` or `enterprise`	usually unset (auto-detected)	usually unset
`DHFW_PQ_MAP`	Community only: PQ name to `host:port`, `"self"`, or `{host, port, auth_type, auth_token_env, use_tls}` with `auth anonymous, basic` or `psk`	e.g. `{"RiskLimits": "localhost:10001"}`	not used
`DHFW_DATA_ROOT`	Community only: folder for emulated `db` tables	e.g. `./dhfw_data`	not used
`DHFW_REPLAY_DATE`	Community only: replay that day's partition as live	e.g. `2026-10-20`	use an Enterprise replay query instead
`DHFW_CONTROLLER_ROOT`	Community only: local checkout of the controller repository	path	not used

Variable	Meaning	Laptop (Community)	Plant (Enterprise PQ)
`DHFW_PLANT_CORE_VERSION`	the plant's Core version; `runtime()` warns on a mismatch	e.g. `41.1`	not needed
`DHFW_USER`	identity on Community	your name	not used (Enterprise identity)

Where a `{"pq": ...}` table resolves. This is tested: one `desk_limits` entry produced each of these as the location variables changed.

The app runs in	Binding	Behaviour
the PQ that owns the table	the global in this worker	in-process; the same update graph; no copy
another PQ on the same plant	`remote_table.in_local_cluster(...).subscribe()`	Barrage subscription; the owner's clock plus yours
a PQ on another plant	`remote_table.for_remote_cluster(url)` with a key file or password	Barrage across plants; needs plant credentials
your laptop (Community)	`dhfw` maps the PQ name to a local server or `"self"`	stand-ins for the real PQs

Two consequences of deploying into the owner's PQ:

- The table becomes local and **consistent with your other tables at every step**, which it was not as a remote subscription.
- The app's code does not change.

Running locally: write an `app.d/*.app` file pointing at your script (see `examples/mm_app.d/`), then run `server/run_server.py --app-dir <dir>`.

Running on the plant: the PQ script imports the app module from the controller repository through `dhfw.meta_import`, sets `DHFW_PLANT` and `DHFW_PQ`, and calls `app.start("live")`. Use an Enterprise replay query for history at a simulated speed.

Cookbook

Add a live feed from any client SDK. Declare a pushed source, then call `feed` from the SDK's callback thread:

```
app.source("quotes", schemas.QUOTES, history=lambda:
db.historical_table("MarketUs", "QuoteStock"))
# in your Solace, websocket or FIX callback:
app.feed("quotes", new_table([...]))          # rows appear at the next cycle
```

Read a plant table. Add a `db` entry to the catalog, then use it as a source:

```
catalog.entries["quotes_us"] = {"db": ["MarketUs", "QuoteStock"]}
app.source("quotes", catalog="quotes_us")
```

On a laptop, put parquet files under `DHFW_DATA_ROOT/MarketUs/QuoteStock/Date=YYYY-MM-DD/`.

Subscribe to another PQ's table. Add an entry with `pq`, `table` and `plant`. To keep the transfer small, pass `subscribe_kwargs` with `filters` and `included_columns`. On Community, dhfw shapes the table on the remote server with pydeephaven, so only those rows and columns travel (tested). On Enterprise they go to remote_table's subscribe; whether filtering happens upstream there is not yet verified.

```
catalog.entries["desk_risk"] = {"pq": "RiskEngine", "table": "desk_risk",
"plant": "ny"}
rates = catalog.resolve("desk_risk", {"filters": ["Desk == `Rates`"],
"included_columns": ["Desk", "Gross"]})
```

Consume a remote server's ticking table from a local app, without `dhfw`. This is what `dhfw` does underneath on Community, and it is tested:

```
from deephaven.barrage import barrage_session
from pydeephaven import Session
from pydeephaven.ticket import SharedTicket

bs = barrage_session(host="riskhost", port=10000,
auth_type="io.deephaven.authentication.psk.PskAuthenticationHandler",
                     auth_token=os.environ["RISK_PSK"])
orders = bs.subscribe(b"s/mm_orders")                # whole table, ticking
in the local graph

s = Session(host="riskhost", port=10000,
auth_type="io.deephaven.authentication.psk.PskAuthenticationHandler",
            auth_token=os.environ["RISK_PSK"])
shaped = s.open_table("mm_orders").where("Sym == `AAPL`").view(["Sym",
"OrderQty"])
ticket = SharedTicket.random_ticket()
s.publish_table(ticket, shaped)                      # keep BOTH s and
shaped referenced
aapl = bs.subscribe(ticket.bytes)                    # only AAPL, 2 columns,
ticking
```

Three lifetime rules, all tested:

- A subscription made before the `pydeephaven` session closes keeps ticking afterwards.
- A new subscription to that ticket fails once the session is closed.
- If the published `pydeephaven` Table object is garbage-collected, the ticket stops resolving even while the session is open.

`deephaven.barrage` or `pydeephaven-ticking`? Inside a Deephaven server (a local app), use `deephaven.barrage`. The result is a table in your graph that you can join, aggregate and put on a dashboard. Use `pydeephaven-ticking` only in a plain Python process with no Deephaven engine, such as an asyncio service. It delivers per-update callbacks (added, removed and modified rows), and you keep the state yourself. Its pip wheels are Linux x86_64 only; Windows needs a source build of the C++ client.

Backtest a day. Pushed sources need `history=`:

```
app = make_app(catalog, quotes_history=lambda:
db.historical_table("MarketUs", "QuoteStock").where("Date == `2026-10-20`"))
app.start("backtest"); print(app.run_history(bucket_ms=1000));
print(app.table("risk"))
```

Persist results.

```
store.snapshot(app.table("risk"), "Risk", "EodRisk")                 # end of
day
store.daily_partition(app.table("book"), "Risk", "DailyBook")      # one
partition per day
with store.intraday_log(app.table("orders"), "Risk", "Orders"):  # live log
other PQs read
    ...
```

Give the desk editable limits. Create a state table and surface it. The dashboard grid edits it in place:

```
limits = store.state_table("Risk", "DeskLimits", prototype, keys=["Desk"])
catalog.override("desk_limits", limits)      # or publish it from the
RiskLimits PQ
```

Customize the dashboard. Use the building blocks inside your own deephaven.ui layout:

```
@ui.component
def my_view():
    return ui.column(ui.row(ui.panel(kpi_row(app.table("risk").view(["Gross",
"PnL"]).sum_by(), ["Gross", "PnL"]))),
                     ui.row(table_panel(app.table("risk"), "Desk risk"),
status_panel(app)))
my_dashboard = ui.dashboard(my_view())
```

Test a stage. No sleeps are needed:

```
app = make_app(catalog).start("test")
app.feed("quotes", one_quote("AAPL", 99.99, 100.01)); app.step()
assert app.table("orders").size == 1
```

Write your own recipe. It is a function from tables to tables, built from table operations:

```
def vwap(trades, by="Sym"):
    return trades.update("Notional = Px * Qty") \
                 .agg_by([agg.sum_(["Notional", "Volume = Qty"])], by=by) \
                 .update("Vwap = Notional / Volume")
```

Limits, open items and evidence

Everything below the Enterprise boundary was tested on Community 42.6. Nothing has run on a plant yet, so the first task on Enterprise is to run the two test suites there.

Limits:

- **Frozen sources.** Backtest and test modes freeze catalog and `live=` sources, so a backtest does not replay limit changes or remote updates over time.
- **Backtest speed.** `run_history` pulls history into Python (pandas) to slice it. Fine for a day of quotes per symbol set, but for very large histories use an Enterprise replay query.
- **Community emulation.** On Community, `state_table` and `intraday_log` live in memory and vanish on restart. Remote subscriptions have no reconnect policy, and the pydeephaven sessions that shape them stay open until dhfw.remote.close_all(). ACLs are not enforced.
- **No laptop-to-plant client binding yet.** Developing on a laptop against the real plant's tables needs the Core+ client and credentials; today remote names map to local stand-in servers.
- **Simulated fills only.** Recipes give the trading skeleton and a cross-the-spread fill model, not a full order-management system.

Open items:

- [ ] Run `tests/test_dhfw_community.py` and `examples/07_app_framework.py` on a Core+ worker with `DHFW_RUNTIME=enterprise`
- [ ] Confirm the `auth_context` function name and `for_remote_cluster` credentials on the plant
- [ ] Add a laptop-to-plant binding through the Core+ Python client
- [ ] Persist Community `state_table` contents to parquet so local edits survive restarts

Evidence:

Claim	Test	Result
Live and backtest give identical results, equal to the hand-written engine	`examples/07_app_framework.py`	8 of 8 checks; risk equal to rung 6 to the cent

Claim	Test	Result
A failing action is supervised	same	1 error in 1 call recorded; routing continued
The catalog resolves by location	same	local global, same plant and another plant from one entry
Storage recipes round-trip	same	snapshot 1 row; daily partitions 2 rows
The dashboard renders and edits	`examples/mm_app.d/mm_live.py`, browser	a limit edit cleared a breach at the next cycle
`dhfw` emulations	`tests/test_dhfw_community.py`	12 of 12
A local app gets ticking remote tables, shaped on the remote server, over PSK auth	tests/probe_remote_ticking.py, tests/test_dhfw_community.py	4 of 4 and 12 of 12; filtered rows only, 2 to 7 rows in 2 s
Shared-ticket lifetimes	same	existing subscription survives session close; new one fails; dropping the published Table breaks the ticket

Code: `dhapp/` and `dhfw/` in [taymarti/deephaven-framework-study](). Background on the engine itself: the Programmer's Guide.